ТЕМА 5. АДАПТИВНАЯ ВЁРСТКА

ЛЕКЦИЯ 5.8. ГИБКИЕ МЕДИА.

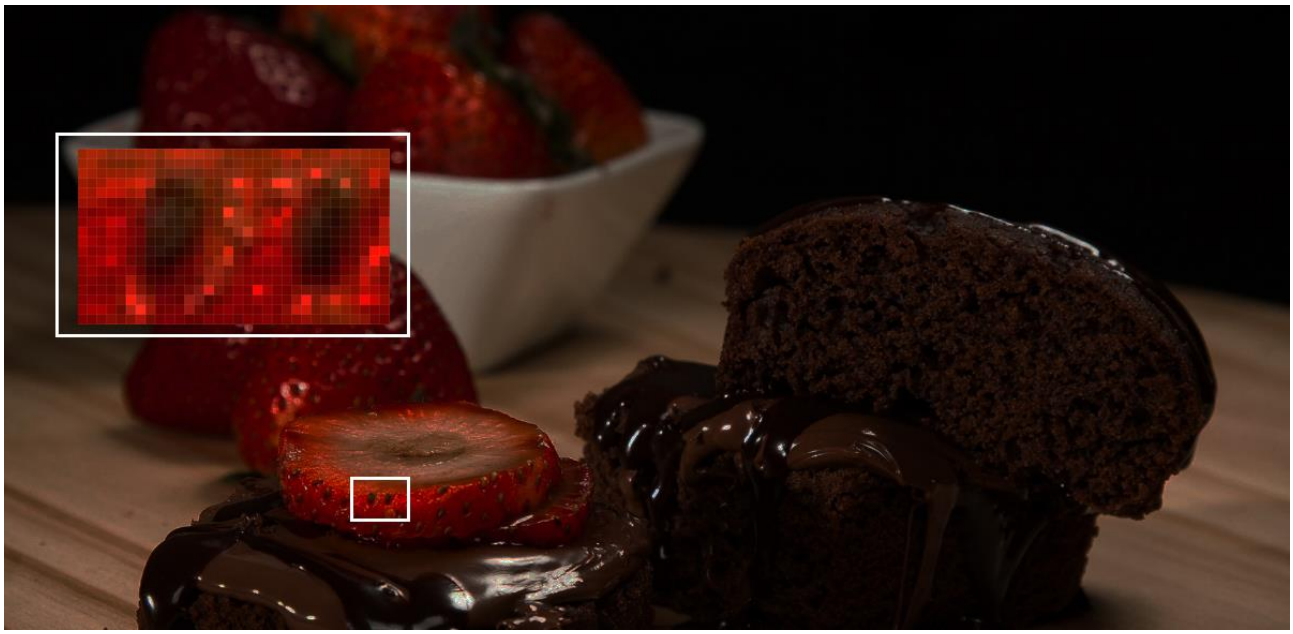
Введение. Атрибуты тега img. Атрибуты srcset и sizes. Функция imgset(). Тег Picture.

Введение

Знакомимся с третьей составляющей адаптива — с гибкими изображениями — такими, которые отображаются хорошо на устройствах с сильно отличающимися размерами экрана, разрешением, и прочими характеристиками и рассмотрим инструменты, которые имеются в HTML для их реализации.

Итак. Все изображения, которые мы используем на страницах, можно условно разделить на 2 группы:

Фоновые изображения – это шапки страниц, для которых важно сохранение качества при изменении размера. Вспоминаем основы растровой графики – изображения одинаково теряют в качестве как при уменьшении, так и при увеличении масштаба.



Контентные изображения – изображения товаров, фото сотрудников, все то, что мы размещаем в блоки страницы. А вот

©Pechora-PRO

здесь проблема в другом… Для них приоритетом станет возможность показать даже на маленьком экране действительно значимые части изображений. Например, лицо сотрудника на мобильном взамен портрета в полный рост на десктопе.





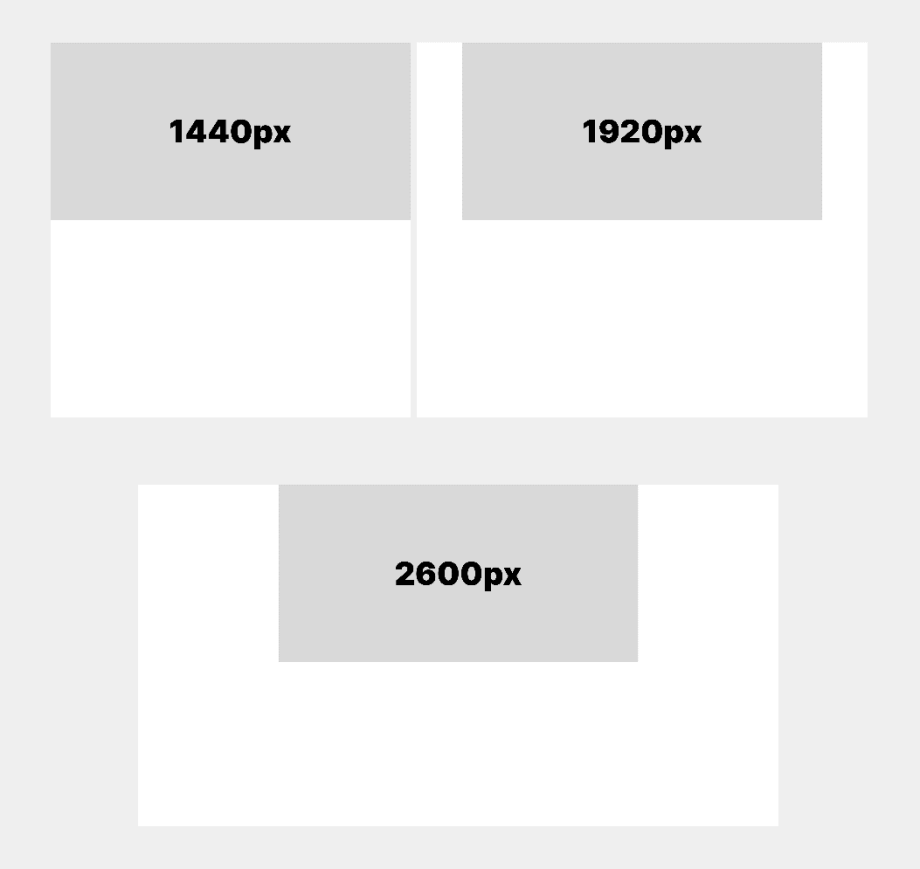
Как видите задачи стоят разные, значит и средствами будем пользоваться разными.

Атрибуты тега img

Итак… Загружая на страницу фоновые изображения мы должны представлять себе, как они буду себя вести при изменении размера экрана.

При увеличении решение может быть простым… Ограничим максимальную ширину контента значением ширины фонового изображения. При увеличении размера экрана получим поля слева и справа. Или же заранее подготовим для использования изображение размером 4к+

©Pechora-PRO



А вот что делать при открытии страницы на маленьком экране? Конечно, можно подобрать и центрировать изображение, так чтобы значимая его часть всегда попадала в поле зрения, а лишняя обрезалась, но проблема избыточного трафика никуда не денется…

И вообще наша задача адаптировать изображения, а не адаптировать страницу под них.

Знакомый нам элемент **img** встраивает изображение в документ и имеет обязательный атрибут **src**. Он содержит путь к изображению, которое мы собираемся встроить в страницу. Один элемент – одна ссылка – одно изображение.

©Pechora-PRO

```
<div class="images" id="img">
    <img class="image"
        <!-- адрес изображения -->
        src="/media/images/img_1.jpg"
        <!-- замещающий текст -->
        alt="Описание изображения">
</div>
```

В отличии от него атрибут **alt** обязательным не является, но содержит текстовое описание изображения. Кстати, это описание очень полезно для адаптации, только несколько другой — программы чтения с экрана озвучат это описание пользователям, если же изображение не может быть загружено по какой-либо причине, то текст заменит собой изображение.

Эти два атрибута нам знакомы, но сегодня нас интересуют другие.

Атрибуты srcset и sizes

Это два новых для нас атрибута — **srcset** and **sizes** — позволяющих добавить дополнительные изображения с пометками, чтобы браузер выбрал подходящее.

srcset включает названия изображений, среди которых браузер выберет нужное и их размеры.

- Содержит список возможных источников изображений для использования.
- Источники записаны в строки и разделены запятыми
- Строки состоят из набора элементов:
    - Адрес изображения.
    - Дескриптор ширины – физическая ширина картинки в пикселях; целое положительное число, за которым следует **w**, например 600w, 1000w.

©Pechora-PRO

- Дескриптор плотности пикселей – плотность пикселей устройства (display density) - DPR (device pixel ratio), положительное десятичное число, за которым следует **x**, например 2x, 3x.

Дескриптор нужен браузеру, чтобы он мог принять решение какое изображение из списка загружать (ведь он заранее ничего не знает о картинке, которая находится по ссылке). А с дескриптором, браузер заранее понимает какая у изображения ширина или для какого DPR она подходит.

```
<div class="images" id="img">
    <img srcset="img-320w.jpg 320w,
                 img-480w.jpg 480w,
                 img-800w.jpg 800w"
         sizes="..."
         src="img-800w.jpg" alt="Описание изображения">
</div>
```

адрес изображения дескриптор ширины

Какой изображение выберет браузер в нашем примере?

Если у вашего устройства DPR=1:

- image-320.jpg - если ширина экрана > 0px и <= 320px.
- image-480.jpg - если ширина экрана > 320px и <= 480px.
- image-800.jpg - если ширина экрана > 480px.

Если у вашего устройства DPR=2 (подходящая картинка будет в 2 раза больше её физической ширины):

- image-320.jpg - если ширина экрана > 0px и <= 160px (320/2).
- image-480.jpg - если ширина экрана >160px и <=240px (480/2).
- image-800.jpg - если ширина экрана >240px.

©Pechora-PRO

Дескриптор ширины в некоторых случая можно заменить дескриптором плотности.

```
<div class="images" id="img">
    <img
        srcset="img-320w.jpg,
                img-480w.jpg 1.5x,
                img-640w.jpg 2x"
        src="img-640w.jpg" alt="Описание изображения">
</div>
```

адрес изображения дескриптор плотности

Если никакой дескриптор не задан, то по умолчанию он равен 1x.

sizes

Поскольку браузер ничего не знает о внешнем виде страницы, он будет считать, что изображение занимает всю ширину экрана. Но если оно занимает лишь часть, то в результате будет загружено изображение размером больше, чем нужно.

Логично предположить, что у браузера в распоряжении есть стили и он может узнать какая ширина в итоге будет у изображения, НО, на момент парсинга тега IMG стили еще не интерпертированы, макет еще не нарисован! Если браузер будет ждать, пока все таблицы стилей будут разобраны и выполнены, это приведет к задержке загрузки изображений.

Если для изображения не указаны никакие размеры в css, то **sizes** ограничит его размер также, как это делает атрибут **width**.

sizes содержит список элементов (через запятую), где каждый элемент описывает размер изображения по отношению к ширине экрана (области просмотра).

©Pechora-PRO

Использование **sizes** дает браузеру дополнительную информацию, чтобы правильно выбрать подходящее изображение из **srcset** и начать его загрузку, как только он увидит элемент **img**, не дожидаясь, парсинга стилей.

```
<img
   srcset="small.jpg 300w,
           medium.jpg 600w,
           large.jpg 900w"

   sizes="(max-width: 300px) 100vw,
          (max-width: 600px) 50vw,
          (max-width: 900px) 33vw,
           900px"
   src="image.jpg">
```

Обратите внимание, что у последнего размера нет медиа-условия — это значение будет принимать слот изображения по умолчанию. Оно будет использоваться, когда ни одно из предыдущих медиа-условий не подошло.

Важно! Будьте внимательнее к порядку объявления проверок, потому что браузер игнорирует все следующие проверки после первой успешной.

Элементы в **sizes** могут иметь:

- **медиа-условие** - например, (max-width: 300px) - оно описывает ширину области просмотра и означает, что указанный размер слота подходит для ширины экрана от 0 до 300 пикселей (включительно). Это похоже на медиа-запрос, но с некоторыми ограничениями. Вы не можете использовать screen или print.
- Ширина слота изображения для указанного медиа-условия. Для указания ширины можно использовать разные величины em, ex, ch, rem, vw, vh, vmin, vmax, cm, mm, q, in, pc, pt, px, но **НЕ проценты**…

©Pechora-PRO

Например, в таком коде

```
<img
   srcset="w-225.jpg 225w,
           w-300.jpg 300w,
           w-350.jpg 350w,
           w-640.jpg 640w"

   sizes="(max-width: 400px) 100vw,
          (max-width: 700px) 50vw,
          (max-width: 900px) 33vw,
          225px"
   src="image.jpg"
>
```

- Если ширина области просмотра превышает 900px, слот изображения занимает фиксированную ширину 225px.
- От 700 до 900px занимает 33vw, т.е. 33% от ширины окна просмотра.
- От 400 до 700px, занимает 50vw, т.е. 50% от ширины окна просмотра.
- До 400px, слот изображения занимает 100vw, т.е. всю ширину окна просмотра.

На основе этих данных браузер выберет одно из изображений **srcset** и загрузит его в определенную область.

Итог

В итоге браузер при построении страницы сделает следующее:

1. Посмотрит на ширину экрана устройства.

2. Попытается определить подходящее медиа-условие из списка в атрибуте **sizes**.

3. Посмотрит на размер слота к этому медиа выражению.

4. Загрузит изображение из списка из **srcset**, которое имеет тот же размер, что и выбранный слот, или, если такого нет, то

©Pechora-PRO

первое изображение, которое больше размера выбранного слота.

Старые браузеры, не поддерживающие эти возможности, просто проигнорируют их и возьмут изображение по адресу из атрибута **src**.

Функция imgset()

image-set() - это функция CSS, которая позволяет вам сделать изображение отзывчивым, показывая варианты одного и того же изображения для разных разрешений экрана, когда это необходимо. Эта функция работает почти также, как атрибут **srcset** для тегов **img**.

Задайте с помощью функции значение свойства background-image и сделайте фон адаптивным.

Вы можете выбрать изображение в зависимости от плотности пикселей устройства.

```
/* Вот так */
div{
    background-image:
        image-set(
         "image1.jpg" 1x,
         "image2.jpg" 2x
         );
}

/* или так */
div{
    background-image:
        image-set(
            url("image1.jpg") 1x,
            url("image2.jpg") 2x
        );
}
```


©Pechora-PRO

Или задавать разные варианты градиента:

```
div{
    background-image:
        image-set(
            linear-gradient(white, blue) 1x,
            linear-gradient(blue, green) 2x
        );

}
```

Еще один способ – адаптивность в зависимости от поддерживаемых форматов.

```
div{
    background-image:
    image-set(
        url("image1.avif") type("image/avif"),
        url("image2.jpg") type("image/jpeg")
      );
}
```

Важно! Не все браузеры поддерживают возможность использования функции, а значит необходимо предусмотреть альтернативный вариант.

```
div {
    background-image: url("image2.jpg");
    background-image: image-set(
      "image1.avif" type("image/avif"),
      "image2.jpg" type("image/jpeg")
    );
}
```

Picture

Элемент **picture** по аналогии с **audio** и **video** позволит внедрить в нашу страницу изображение.

©Pechora-PRO

В отличии от **img** мы не будем ограничены форматом внедряемого изображения. Теперь нам доступна для внедрения и векторная графика и изображения формата **webp**.

Итак, HTML-элемент **picture** служит контейнером для одного или более элементов **source** и одного элемента **img** для обеспечения загрузки оптимальной версии изображения для различных размеров экрана.

```
<picture>
  <source media="(min-width: 650px)"
  srcset="img_big.jpg">
  <source media="(min-width: 465px)"
  srcset="img_medium.jpg">
  <img src="img_small.jpg" alt="Picture">
</picture>
```

Браузер рассмотрит каждый из дочерних элементов **source** и выберет один, соответствующий лучшему совпадению.

Если совпадений среди элементов **source** найдено не будет, то будет выбран файл, указанный атрибутом **src** элемента **img**.

Затем выбранное изображение будет загружено в пространство, отведенное элементу **img**.

source

- srcset - определяет URL-адрес изображения для отображения.
- media - определяет медиавыражение, которое браузер будет анализировать для выбора элемента source. Если выражение определяется как ложное, то элемент source пропускается.

©Pechora-PRO

```
<!-- три варианта загружаемого изображения -->
<picture>
  <!-- на экране больше 650px -->
  <source media="(min-width: 650px)"
  srcset="img_big.jpg">
  <!-- на экране от 465px до 650px -->
  <source media="(min-width: 465px)"
  srcset="img_medium.jpg">
  <!-- если тег не поддерживается или ни одно из
  условий не подходит -->
  <img src="img_small.jpg" alt="Flowers">
</picture>
```

- **type** - позволяет обозначить MIME-тип данных, указанных в атрибуте **srcset** элемента **source**. Если браузер не поддерживает данный тип, то элемент **source** опускается.
- **sizes** - указывает допустимые относительные размеры источника.

```
<!-- предлагаем к загрузке три типа изображений в
форматах avif, webp и png -->
<picture>
  <!-- сначала браузер проверяет поддержу в
  формате avif -->
  <!-- если формат поддерживается, то загружаем
  текущую картинку -->
  <!-- если нет, то идем дальше -->
  <source srcset="img.avif" type="image/avif">
  <!-- аналогично с форматом webp -->
  <source srcset="img.webp" type="image/webp">
  <!-- и если ни один из предложенных форматов не
  поддерживается, загружаем img.png -->
  <img src="img.png" alt="image">
</picture>
```

Так чем же глобально это отличается от того, что делает союз img+srcset+size? На самом деле использование picture не будет

©Pechora-PRO

раскрывать все свои возможности, если мы хотим везде загружать одно и то же изображение формата jpg или png…

А вот если нашей задачей становится так называемое художественное изменение… например, изменение соотношения ширины и высоты изображения на каком-то определенном брейкпоинте или обрезка изображения, чтобы увеличить масштаб на мобильном устройстве, тогда без него сложно…

В заданиях к лекции пример смены ориентации графики и смены разрешения при помощи тега **picture, атрибутов srcset и sizes**. Скачивайте, смотрите, редактируйте.

Кстати, в документации заявлена полная поддержка этих возможностей браузерами (за исключением разве что IE). Но, что интересно, при работе в Google Chrome может возникнуть проблема... контейнер для изображения будет уменьшаться в соответствии с заданными правилами, а вот картинка подгружаться одна и та же.

Проблема в том, что Chrome кеширует картинку. И если первый раз загружено одно изображение, то браузер будет его показывать всегда.

Решить проблему можно жесткой перезагрузкой с очисткой кэша на нужном разрешении, переходом в режим ИНКОГНИТО или сменой браузера. Например, Firefox Developer Edition подгружает нужную картинку на лету.

©Pechora-PRO